

HW2: Binary Exponentiation

MIPs Training

- If-then-else
- Loop instructions

Speaker: 王睿杰

Keyword: *Mars, MIPs*

Sep.15, 2025

wri651121@gmail.com

HW Target

```
#include <iostream>

int main() {
    int base, exponent;

    while (true) {
        // --- 1. Read Inputs ---
        std::cout << "base: ";
        std::cin >> base;

        std::cout << "exponent: ";
        std::cin >> exponent;

        // --- 2. Handle Edge Cases ---
        if (base == 0 && exponent == 0) {
            std::cout << "Both base and exponent are zero. Exiting." <<
std::endl;
            break;
        }

        if (exponent < 0) {
            std::cout << "Error: Exponent cannot be negative.\n\n";
            continue;
        }

        // --- 3. Binary Exponentiation Algorithm ---
        int power_result = 1;
        int temp_base = base;
        int temp_exponent = exponent;

        while (temp_exponent > 0) {
            if (temp_exponent & 1) {
                power_result *= temp_base;
            }
            temp_base *= temp_base;
            temp_exponent >>= 1;
        }

        // --- 4. Print the Power Calculation Result ---
        std::cout << "result: " << power_result << std::endl;

        // --- 5. Hamming Weight Calculation (using a for loop) ---
        int weight_count = 0;
        for (int i = 0; i < 32; ++i) {
            if ((power_result >> i) & 1) {
                weight_count++;
            }
        }

        // --- 6. Print the Hamming Weight Result ---
        std::cout << "hamming weight: " << weight_count << std::endl;
        std::cout << std::endl;
    }
    return 0;
}
```

Translate

MIPs code

Simulate

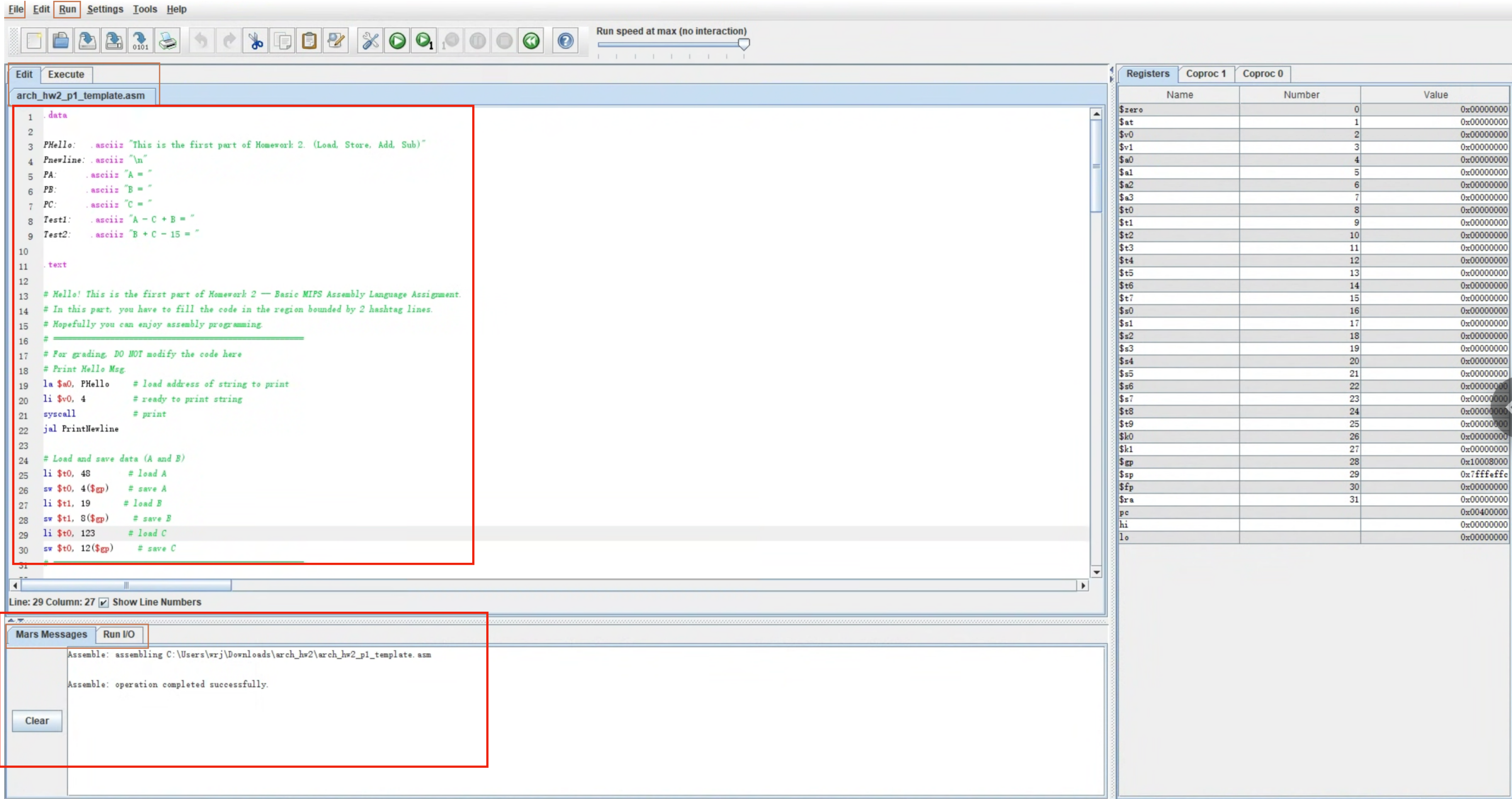


Mars4_5.jar

MIPs Simulator: Mars

Your MIPs code

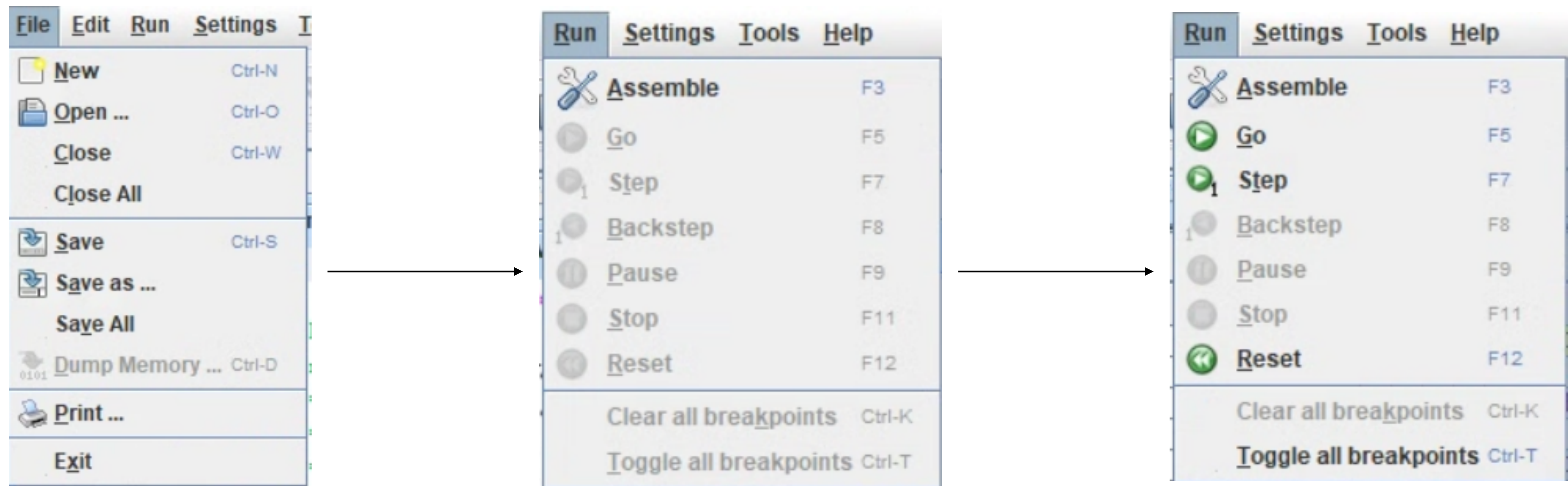
Result



Main Interface

MIPs Simulator: Mars

To run the code:



Save

Assemble

Go

Binary Exponentiation: I/O

Objective: Efficiently Calculate $base^{exponent}$

Input:

- base: the number to be exponentiated (a **signed integer**, $-2^{31} \leq base < 2^{31}$).
- exponent: the power (a **signed integer**, $-2^{31} \leq exponent < 2^{31}$).

Output:

- result: The final result $base^{exponent}$ (a **signed integer**, $-2^{31} \leq result < 2^{31}$, overflow handling is not required).
- hamming weight: the total count of '1's in binary representation of the result (a **signed integer**, $0 < hammingweight \leq 32$).

Requirement:

- If the **exponent** is negative, display the error message:

Error: Exponent cannot be negative.

- If both the **base** and **exponent** are zero, terminate the program and display:

Both base and exponent are zero. Exiting.

Binary Exponentiation: Example

Calculate: 3^2

Input:

base: 3

exponent: 2

$$3^2 = 9 = 1001_2$$

Output:

result: 9

hamming weight: 2

Invalid Calculate: 2^{-5}

Input:

base: 2

exponent:-5

Output:

Error: Exponent cannot
be negative.

End program

Input:

base: 0

exponent:0

Output:

Both base and exponent
are zero. Exiting.

Binary Exponentiation: MIPS instruction you need to know

Print/Scan Instructions

- *la (load address)*: Loads the address of a memory location or label into a register.
- **Syscall**: Triggers a system call, allowing the program to request specific services (e.g., input/output) from the operating system based on the value in register \$v0, for more system call functions, please refer to the IV. Appendix.

E.g. Please refer to **2025_arch_hw2_spec.pdf**

Arithmetic Instructions

- *mul (multiply)*: Perform multiplication.

E.g.

```
# Perform the multiplication
mul $t2, $t0, $t1      # Multiply $t0 and $t1, store the result in $t2
```

Binary Exponentiation: Concept

Objective: base = 5, exponent = 13, Efficiently Calculate 5^{13}

Normal method: $5^{13} = \underbrace{5 \cdot 5 \cdot 5 \cdot 5 \cdot 5}_{\text{6 multiplications}} \cdot \underbrace{5 \cdot 5 \cdot 5 \cdot 5 \cdot 5}_{\text{6 multiplications}}$

The Core Idea: Decomposing the Exponent

Notice that: $5^{13} = 5^{1101_2} = 5^8 \cdot 5^4 \cdot 5^1$ 3 multiplication

1. We can use $base \cdot base$ to iterative construct $5^1, 5^2, 5^4, 5^8$

Initial: $base_0 = 5^1$

Iteration 1: $base_1 = base_0 \cdot base_0 = 5^1 \cdot 5^1 = 5^2$

Iteration 2: $base_2 = base_1 \cdot base_1 = 5^2 \cdot 5^2 = 5^4$

Iteration 3: $base_3 = base_2 \cdot base_2 = 5^4 \cdot 5^4 = 5^8$

2. We can use the binary form of exponent (1101_2) with right shift instruction to choose $5^1, 5^4, 5^8$

If $LSB = 1$: $result = result \cdot base$

Binary Exponentiation: basic algorithms

Initialize: $\text{Result} = 1, \text{base} = a$

For each bit in the binary exponent:

1. If the bit is 1, $\text{Result} = \text{Result} * \text{base}$

2. $\text{base} = \text{base} * \text{base}$

Repeat until all bits are processed.

```
def binary_power(base, exponent):  
    result = 1  
    while exponent > 0:  
        if exponent & 1:  
            result = result * base  
        base = base * base  
        exponent >>= 1  
    return result
```

Binary Exponentiation: Hint

You can translate this c++ code directly:

```
#include <iostream>

int main() {
    int base, exponent;

    while (true) {
        // --- 1. Read Inputs ---
        std::cout << "base: ";
        std::cin >> base;

        std::cout << "exponent: ";
        std::cin >> exponent;

        // --- 2. Handle Edge Cases ---
        if (base == 0 && exponent == 0) {
            std::cout << "Both base and exponent are zero. Exiting." <<
std::endl;
            break;
        }

        if (exponent < 0) {
            std::cout << "Error: Exponent cannot be negative.\n\n";
            continue;
        }

        // --- 3. Binary Exponentiation Algorithm ---
        int power_result = 1;
        int temp_base = base;
        int temp_exponent = exponent;

        while (temp_exponent > 0) {
            if (temp_exponent & 1) {
                power_result *= temp_base;
            }
            temp_base *= temp_base;
            temp_exponent >>= 1;
        }

        // --- 4. Print the Power Calculation Result ---
        std::cout << "result: " << power_result << std::endl;

        // --- 5. Hamming Weight Calculation (using a for loop) ---
        int weight_count = 0;
        for (int i = 0; i < 32; ++i) {
            if ((power_result >> i) & 1) {
                weight_count++;
            }
        }

        // --- 6. Print the Hamming Weight Result ---
        std::cout << "hamming weight: " << weight_count << std::endl;
        std::cout << std::endl;
    }
    return 0;
}
```

Don't use NLP model to generate MIPs code

Grading

Grading Criteria

- **Correctness of basic algorithm: 60% = 20% + 20% + 20%**
 - During the evaluation, the simulator's run speed is set to **30 inst/sec**, with a time limit of **5 minutes** for each testcase.
 - 3 random testcase, each 20% (result: 10%, hamming weight: 10%).
- **Correctness of requirement: 20% = 10% + 10%**
 - Negative exponent condition , 1 random testcase.
 - Exiting condition
- **Message format: 20% = 10% + 10%**
 - Input/Output messages format